

2024 Sept OPPE 1 - SET 2

Section 1 - Problem 2

Problem Statement

Problem statement:

- You need to write a function **within_and_has_double_quotes**
- that **checks if a given string starts and ends with double quotes**
- and if there are double quotes inside the string (excluding the first and last characters).
- Return **True** if **both conditions are met**, and False otherwise.
- This is a **function type question**, you don't have to take input or print the output, just have to complete the required function definition.

Example and Explanation:

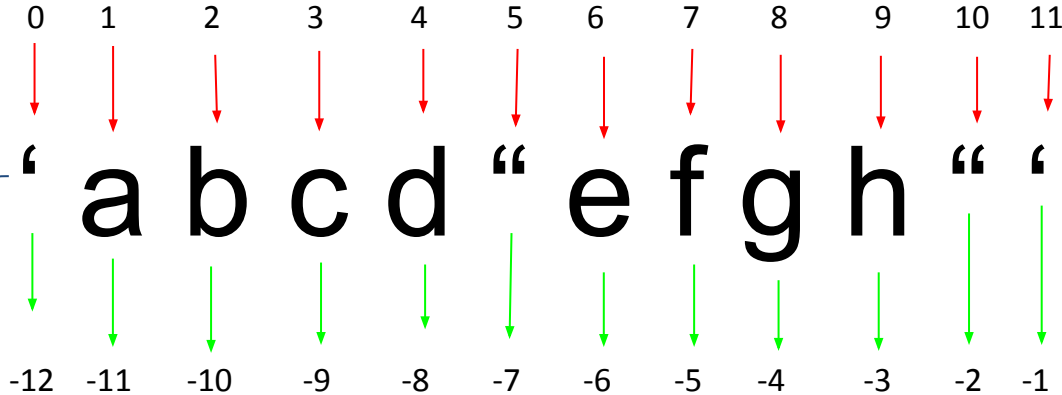
- Let our input be a string
- **s = "abcd"efgh"**
- Condition 1: Check if the string starts with double quotes.
 - **TRUE**
- Condition 2: Check if the string ends with double quotes.
 - **TRUE**
- Condition 3: Check if the string has double quotes inside it (exclude first and last characters)
 - **" + abcd"efgh + "**
 - **TRUE**, there is a double quotes between 'd' and 'e'
- Result : **TRUE**

Concept of Indexing in Strings

Given a string, `s = 'abcd"efgh"'`

We use the concept of indexing here. Index is like a seat number, it tells you exactly where something is. It simply refers to the position of a character in a string.

Positive indexing
(starts with 0)



Negative indexing
(starts with -1)

The 1st character is a single quote, so:
`s[0] != "\""`

The characters are accessed by the syntax: `string_name[index]`. For example: 'd' in our string is accessed by `s[4]` and the **last character is accessed by `s[-1]`** (Use negative indexing when length of string is unknown)

The last character is a single quote, so:
`s[-1] != "\""`

Concept of escape characters

Escape characters are special characters used inside strings to perform special actions, like moving to a new line or adding quotes.

They start with a backslash `\` , Think of `\` as a signal telling Python: “do something special”

Why do we need escape characters?

Sometimes normal characters have special meaning. Escape characters help us Print quotes inside strings, Move text to a new line, Add tabs or spacing.

Escape character	Meaning	Example	Output
<code>\n</code>	New line	<code>"Hello\nWorld"</code>	Hello World
<code>\t</code>	Tab space	<code>"Hello\tWorld"</code>	Hello World
<code>\\</code>	Backslash	<code>"Hello\\World"</code>	Hello\World
<code>\"</code>	Double quotes	<code>"Hello\"World"</code>	Hello"World
<code>\'</code>	Single quotes	<code>"Hello\'World"</code>	Hello'World

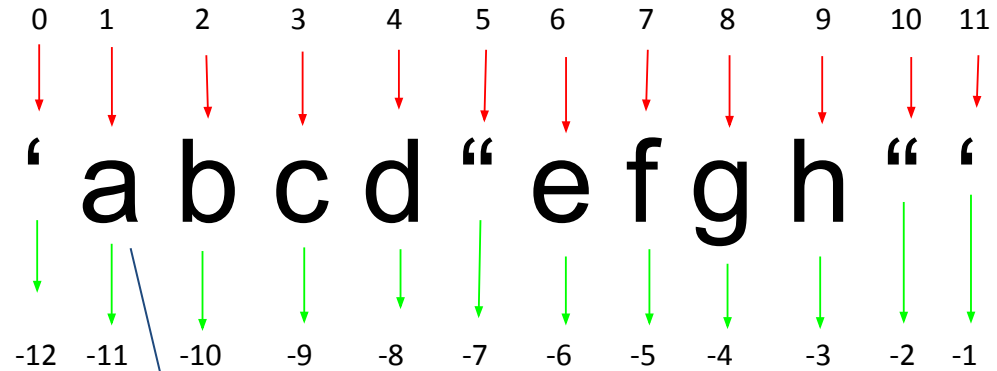
Logic Discussion

So our string is 'abcd"efgh"'

The Logic:

- Check if the **1st and last character** are double quotes first.
 - Use **string indexing** or **built in string functions**
- If True, **iterate** through every character in the string from 2nd character to second last character to **check if double quotes are present inside** string.
 - Use **'for' loops** and **string slicing**
- Return result accordingly.

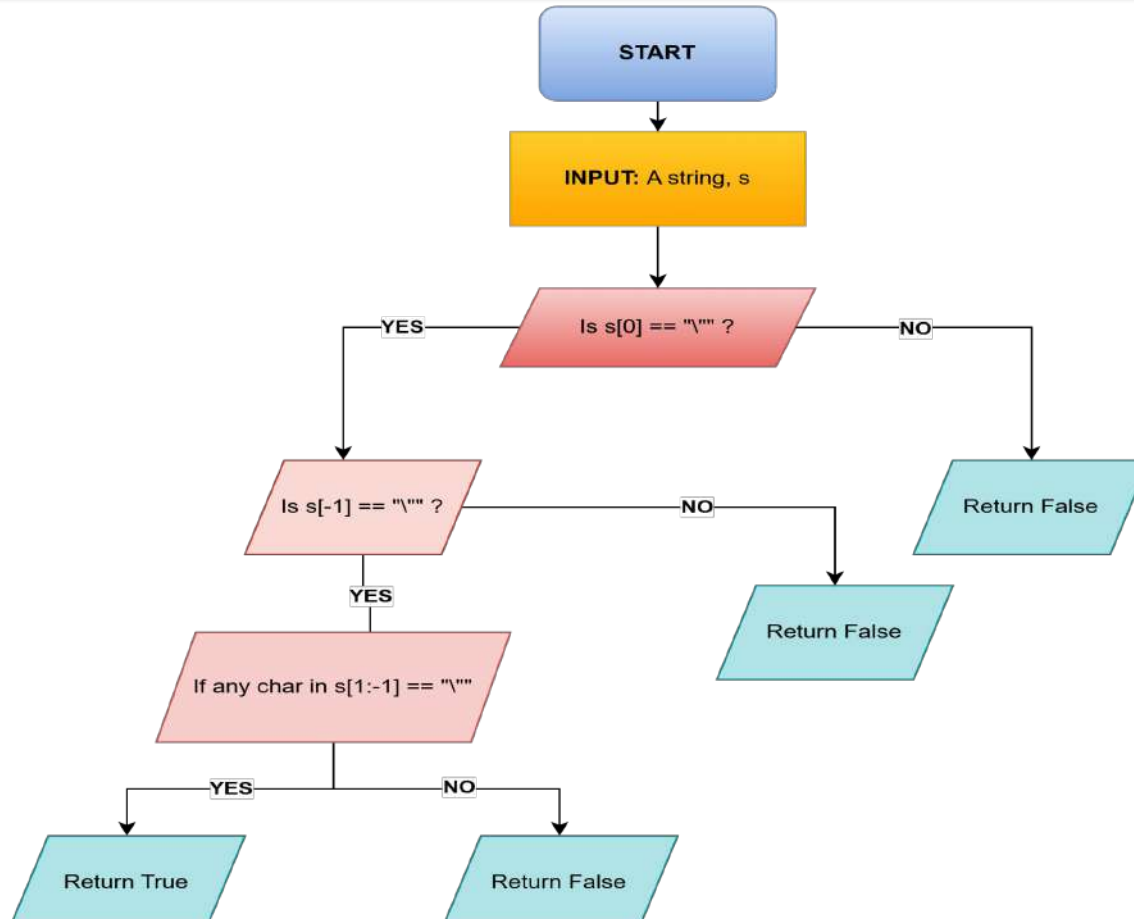
In the string, the 1st character : s[0] itself is not double quotes, so result is False



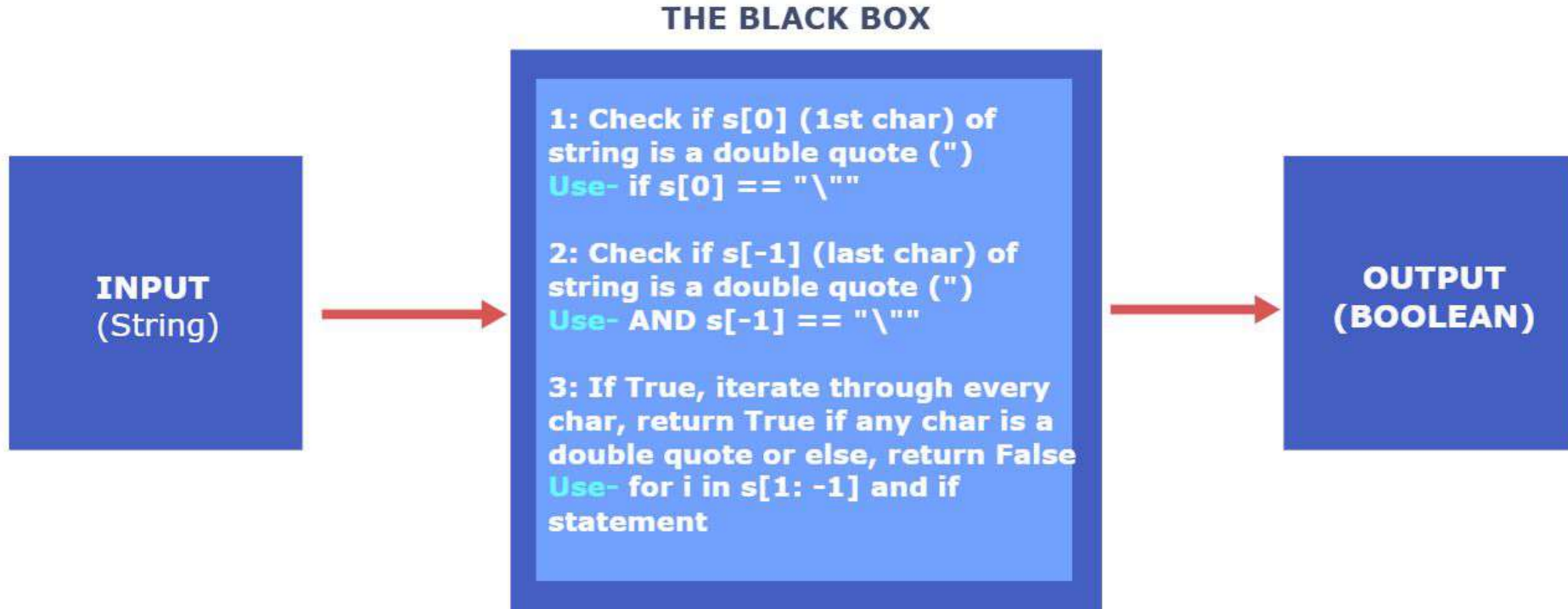
String slicing here: s[1: -1]

string_name [start index: end index],
Note: end index is excluded

Logic Flowchart

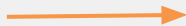


Our Solution Approach



Solution 1: The Explanatory Approach

```
def within_and_has_double_quotes(s):  
    if s[0] == "\"" and s[-1] == "\"":  
        for ch in s[1:-1]:  
            if ch == "'":  
                return True  
        return False
```



Use for loop to iterate over every character in the string.
We can also use single quotes to enclose, instead of using escape characters. ' ' ' is the same as " \" "

A step by step implementation of solution for better understanding

Solution 2: Alternate way using string functions

```
def within_and_has_double_quotes(s):  
    if not (s.startswith('"') and s.endswith('"')):  
        return False  
    return '"' in s[1:-1]
```



Usage of string built in functions:

- `str.startswith(str)` returns True if the string starts the character mentioned in the brackets.
- `str.endswith(str)` returns True if the string ends the character mentioned in the brackets.

An alternative solution using built in string functions in python

Solution 2 - The Concise Approach

```
def within_and_has_double_quotes(s):  
    return s[0] == s[-1] == "\"" and "\"" in s[1: len(s) -1]
```



- This concise one-liner checks the required conditions **in a single return statement instead of using multiple if blocks.**
- It first **verifies that the string starts and ends with double quotes**, and then checks if there is at **least one more double quote inside** the string.
- The **and** ensures both conditions must be true for the function to return True.
- If either condition fails, it automatically returns False.

The Pythonic Optimization: One-liner solution using Chained Comparison.
Result: Identical output, neater code

Solution comparison

Step by Step Approach



- **Higher Readability**
- **Clearer understanding without any missing steps**
- **Best for beginners and debugging**
- **A little time consuming**
- **Best Usage: When problem statement itself is complicated**

Concise One - Liner Approach



- **Neat and shorter code**
- **Fast and time efficient**
- **Best for competitive programming/ when coding with time constraints**
- **Fewer memory assignments**
- **Might get confusing as problem statement gets challenging**
- **Best Usage: for simpler tasks to save time**

Key Takeaways

- 🔑 String Indexing (`s[0]`, `s[-1]`) is used to access the first and last characters of the string.
- 🔑 Chained Comparison checks both starting and ending double quotes in a single expression.
- 🔑 String Slicing (`s[1:-1]`) extracts the substring excluding the first and last characters.
- 🔑 The Membership Operator (`in`) checks whether a double quote exists inside the sliced substring.
- 🔑 The Logical AND Operator (`and`) ensures both conditions must evaluate to `True`.
- 🔑 The function returns a Boolean Value (`True` or `False`) based on the combined logical expression.